Progetto Reti Informatiche A.A. 2023/2024

Vito Pampinella

Codice server

Nella parte server è presente il thread principale, quello del main, che si occupa di gestire lo stdin con l'admin. I comandi sono quelli delle specifiche, più exit. Quando si chiama start [port], il server crea due thread: thread listener e thread timer. Il thread listener crea il socket usando la port e si mette in ascolto per le connessioni. Quando arriva una nuova connessione, crea a sua volta un thread worker che gestisce quell'utente, e si rimette in ascolto. Il thread viene creato e distrutto insieme alla connessione con il client, quindi si usa un tipo di connessione persistente (TCP). Il thread timer ha il compito di gestire i timer. I timer vengono gestiti in modo simile a quello che ci è stato presentato a Calcolatori Elettronici. Quando si chiama stop, il server controlla che non ci siano partite, e in caso disconnette tutti gli utenti connessi (che non sono in partita), eliminando tutte le risorse allocate, e termina il thread listener e il thread timer. Quando si chiama exit, se il programma è fermo, termina.

Codice client

Il codice client è molto semplice. Il codice instaura la connessione con la port passata per argomento, poi crea un thread per gestire i messaggi dal server, mentre il thread principale gestisce lo stdin dell'utente. Tra i due thread avviene un handshake attraverso una condition variable. Quando l'utente finisce di scrivere nello stdin, il messaggio viene mandato tramite socket, e poi il processo principale si mette in attesa di un segnale che la risposta del server sia stata ricevuta dall'altro thread.

Protocollo di comunicazione

Si manda sempre prima un messaggio che rappresenta la lunghezza del messaggio vero e proprio, e poi il testo. Si è deciso di usare text format per semplicità, dato che il traffico di dati non è alto e non richiede una particolare economia sulla quantità di informazioni trasmesse. Sopra ciò, c'è un protocollo di comunicazione tra client e server. Il server può mandare tre tipi di messaggi, distinti dal fatto che iniziano in modo diverso:

- ANSWER@: il messaggio rappresenta una risposta al comando precedentemente mandato dal client, quindi il client può mandare il prossimo comando. Serve a creare un handshake tra client e server.
- INFO@: il messaggio è un semplice messaggio informativo su eventi compiuti da altri giocatori, che non influenza l'handshake tra client e server, e verrà semplicemente stampato in schermo.
- END@: il messaggio è un messaggio di terminazione. Il testo restante verrà stampato in schermo e poi il client terminerà l'esecuzione, facendo chiudere anche a cascata il thread worker dal lato server e le eventuali strutture dati ad esso associate.

Funzionalità aggiuntive

Oltre ai comandi standard per il giocatore, si sono aggiunte le seguenti funzionalità:

- chat message: l'utente può mandare un messaggio agli altri giocatori della partita per comunicare con loro
- drop obj: l'utente può lasciare nella location attuale un oggetto che possiede per liberare spazio o anche per scambiare oggetti con gli altri utenti
- nello scenario 0, quando 2 o più giocatori si trovano nel corridoio, il pavimento inizierà a crollare. Se nessuno si sposta entro 10 secondi, tutti i giocatori perdono la partita. Questo forza i giocatori a collaborare e sincronizzarsi negli spostamenti. Questa funzionalità è stata aggiunta utilizzando i timer e mettendo dei controlli della posizione dei giocatori alla creazione del player, alla sua distruzione e nella chiamata a look [location].

Analisi critica delle scelte progettuali

La scelta di gestire le connessioni utilizzando solo threads è stata fatta per via della velocità di creazione di threads e per la loro facile condivisione di risorse. Si è fissato un numero massimo di giocatori per partita a 10, quindi per la sincronizzazione e mutua esclusione riguardo al gioco stesso non dovrebbero esserci particolari problemi, a prescindere dalla quantità di clients totali. Infatti, una volta entrati in una partita, i thread useranno principalmente mutex sulla singola partita o sui singoli clients collegati alla partita, non interferendo quindi con le altre strutture dati globali. L'applicazione, attualmente, ha un limite massimo di partite e quindi di clients totali, limite che si può cambiare dal file constants.h. Si ritiene comunque che l'applicazione sia sufficientemente scalabile. Gli aspetti negativi dell'applicazione sono più che altro dovuti alla sua complessità per quanto riguarda la mutua esclusione ovvero per l'utilizzo delle risorse che sono condivise da tutti i thread. Probabilmente sarebbe stato più semplice evitare accessi concorrenti utilizzando fork, per esempio con un processo per ogni partita, però si sarebbero dovuti implementare protocolli di comunicazione tra processi, e inoltre la creazione di processi sarebbe risultata in un maggiore overhead per la creazione della partita. Per evitare deadlock, si è utilizzata una gerarchia di mutex. In particolare, per i thread worker, si sono usati principalmente 4 mutex per gestire l'accesso alle risorse condivise, e si è usata la seguente gerarchia per evitare deadlock (1 è il primo su cui fare lock):

1. mutex_clients_array: serve per creare o eliminare clients dal vettore clients
2. mutex_games_array: serve per creare o eliminare games dal vettore games
3. mutex_clients[i]: serve principalmente per leggere o scrivere sul client i-esimo del vettore clients. Va chiamato anche quando un'operazione di aggiunta o rimozione coinvolge la cella i del vettore clients.
4. mutex_games[i]: serve principalmente per leggere o scrivere sul game i-esimo del vettore games. Va chiamato anche quando un'operazione di aggiunta o rimozione coinvolge la cella i del vettore games.

Uno svantaggio di questa implementazione è che non è dinamicamente scalabile. La quantità massima di giocatori va fissata prima dell'avvio, e se si vuole cambiare il limite, l'applicazione va interrotta.